

15. Un compteur qui déborde à 255

Ce n'est pas un bogue : c'est ce que fait un octet.

1. Ce qu'il faut comprendre

`uint8_t` est **un octet** : de 0 à 255. À 255, `compteur++` ramène à 0 — et tout le projet est bâti là-dessus.

L'ordre compte. `image()` est **à la fin** : on dessine, puis on attend. Mettre l'attente en premier ferait sauter la première image.

Laisser tourner quelques secondes suffit à voir le compteur repasser par zéro.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **376 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         25 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireNombre – attend de pouvoir écrire une case
EcrireNombre   33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         160 o ← CopierTuiles – 10 lettres sur 43
```

Lettres gravées : 0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
nombre() × 1    10 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : un compteur qui monte de 1 à chaque image : 0, 1, 2 ... 255,
// puis il repart à 0.
//
// Pourquoi 255 ? Un uint8_t tient sur 8 bits : 256 valeurs, de 0 à 255.
// 255 + 1 ne tient pas : la case « déborde » et revient à 0,
// comme un compteur kilométrique.

#include <nombre> // écrit un nombre en chiffres
```

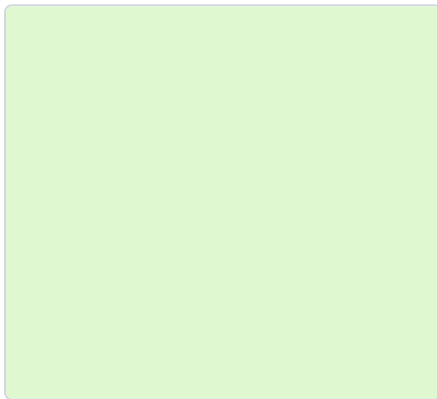
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t compteur = 0;    // Une case « compteur » qui commence à 0.

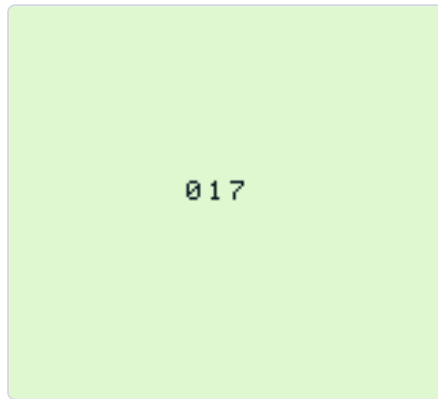
    while (true) {          // Répète pour toujours :
        nombre(8, 8, compteur); // affiche le compteur (3 chiffres),
        compteur++;          // ajoute 1 (« ++ » = « compteur = compteur + 1 »),
        image();             // attend l'image suivante.
    }                       // 60 images par seconde : il fait le tour en 4 secondes environ.
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un nombre qui monte, et qui repart de 000 après 255.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le compteur monte (18 → 58)

✓ il reste dans un octet

✓ et il est affiché

Ensuite : **15.1. La fonction `semer()` — le départ du hasard** — chapitre 3 : Retenir, et réagir. · [gameboy3](#) — écrire une cartouche Game Boy en C++